Autonomous Visual Object Finding with an e-puck: Perception, Route Planning and Safety

Syuan-Hao, Li Isaac Mangano Hyunkyung Lee
Griffith University
s5453352, syuan-hao.li@griffithuni.edu.au
s5418871, isaac.mangano@griffithuni.edu.au
s5432909, hyunkyung.lee@griffithuni.edu.au

Abstract

An e-puck robot in a known Webots arena must find which of eight stations shows a requested target object, and stop at that station's observation position within four minutes without collision. The target identity is available only through a low, 160×120 *camera that sees the dark lower half of each poster. We combine three parts. A DINOv3 ViT-S/16 classifier is fine-tuned on the eight reference images with view-matched augmentation; it reaches 100% accuracy on unseen station frames at near-square poses and runs in 32 ms on a CPU. An A* planner visits stations in greedy nearest-first order on the supplied occupancy grid. A two-layer safety monitor combines directional proximity checks with progress-based stuck detection. Scored by an independent referee, the final controller completes all three provided student missions (130–162 s) and all 192 missions of a Monte Carlo benchmark over every start, target and station assignment (median 73 s, worst 162 s), with no collision and a median stopping error of 3.7 cm. The result is insensitive to the camera scan and to start-pose perturbation. When an unmapped box blocks the route, the safety monitor raises success from 0% to 59%, and its remaining failures point to two concrete fixes.*

1. Introduction

The task is simulated in Webots [5] with an e-puck robot [6]. The arena is a 4×4 m room with five barriers and eight observation stations, each displaying one of eight target objects. A mission names one target; the robot must drive from one of three designated starts to the station that displays it and stop within 0.20 m of that station's observation position, without any collision, within 4:00 of simulation time. The same controller must run unchanged for every start, target and target-to-station assignment.

What makes it hard. The map, station coordinates and robot pose are given, but *which* station holds the target is not: it must be read from the camera, without access to world files or texture names. The camera sits low, so at the observation pose it sees only the bottom half of each poster, and four of the eight stations are about three times darker than the others (Sec. 3). The robot can arrive with a heading error, the barriers carry 20 distractor images, and the corridors leave only centimetres of clearance.

Project idea. We separate *where to go* from *what is there*. Because the map is known, navigation is exact: an A* planner drives the robot cell by cell to each station's observation pose in greedy nearest-first order. Vision is used only at a station, to decide whether its poster is the target. A safety monitor watches every simulation step and triggers replanning when something unexpected happens (Fig. 1). Each part was chosen and validated with its own experiments, and the whole system with a referee-scored Monte Carlo benchmark.

Contributions.

- **Perception:** A study of 14 encoders shows that the first detector's failures came from its training data, not its model. A DINOv3 ViT-S/16 fine-tuned on the eight references with view-matched augmentation reaches 100% on unseen station frames, at 32 ms per frame on a CPU, and its confidence separates its own errors (all $\leq$0.157) from correct predictions. (Sec. 3)
- **Route planning:** Optimal A* search on the 4-connected occupancy grid with greedy nearest-station ordering and rotation-matrix motion execution, compared offline against Dijkstra, greedy best-first and a turn-penalised variant. (Sec. 4, Appendix A)
- **Safety:** A two-layer safety monitor combining per-step directional proximity monitoring with progress-based stuck detection for contacts missed by the proximity sensors, with replanning after each safety event. The front thresholds were empirically calibrated, improving stopping clearance in a 45° approach from 3.6 cm to

5.2 cm, and the stuck detector was validated through fault-injection tests. (Sec. 5)

- **Evaluation:** An independent supervisor referee scores runs against the assignment criteria. We use it on the three student missions, a 96-mission benchmark across four controller versions, and ablations of the scan, the start pose and the safety layers. (Sec. 6)

2. System Overview

Mission flow. The controller reads the target label from `config/assessment_mission.json` and repeats one cycle until it finds the target (Fig. 1). It selects the nearest unvisited station, plans a shortest grid path to that station's observation cell (Sec. 4), and drives the path cell by cell while the safety monitor checks every simulation step (Sec. 5). A safety event stops the move, and the controller replans from the robot's current cell. At the observation cell the robot turns to the station's observation yaw, takes four frames and classifies them (Sec. 3). If the most confident label is the target, the robot stops and holds position; otherwise it moves on to the next station.

Sensors and what they are used for. The camera decides target identity and nothing else. GPS and the IMU, combined with the supplied occupancy grid and station coordinates, give the robot's cell and heading for planning and driving. The proximity sensors ps0–ps7 feed only the safety monitor. The controller never reads world files, texture names or simulator ground truth, and it does not use a supervisor. The supervisor-based referee and capture tools in our evaluation are separate programs that only score runs or build offline test data.

Modules and interfaces. Each concern lives in one package, and the main loop composes them through small interfaces (Tab. 1). Motion functions return a set of safety events, empty on success, so planning and safety stay decoupled: the motion layer stops and recovers, and the main loop decides to replan.

Control loop and timing budget. The controller runs at Webots' basic time step of 32 ms in synchronous mode, so simulation time advances only when the controller calls `step`. Model loading and the 32 ms inference therefore cost wall-clock time but no mission time. Mission time goes almost entirely to motion. A 0.1 m cell takes about 25 steps (0.8 s) at full wheel speed, an in-place 90° turn about 0.7 s, and the station scan about 3.5 s per mission in total (Tab. 8). Across the 96-mission benchmark the final controller needs 73 s at the median and 162 s in the worst case, which uses 68% of the 240 s limit (Sec. 6).

Module	Interface (input → output)
`gps`	sensors → grid cell; heading (multiple of 90°) and yaw
`stations`	robot cell → nearest unvisited (observation cell, yaw)
`path`	start, goal cell → [(cell, global move)] via A*
`robot`	global move → turn + one-cell move; returns safety events
`safety`	ps0–ps7 → danger directions; progress → STUCK
`detector`	camera frames → (label, confidence), best of four

Table 1. Controller packages and the interface each one exposes to the main loop. Motion returns safety events instead of acting on them, which keeps planning, safety and perception independent and testable.

3. Perception

3.1. Problem and design rationale

Perception has one job: decide which of the eight targets the poster at a station shows. The only labelled data is one 512×512 reference image per target. The camera view differs from the references in two ways (Fig. 2). First, the e-puck camera (160×120) sits low, so at the observation pose it sees only the *bottom half* of the poster, with the dark barrier body and the floor below it. Second, stations S1, S2, S3 and S8 are in shadow: their mean poster grey level is about 34, against 100–104 at S4–S7. The classifier must also run on a CPU and must not use world files or texture names.

3.2. Evaluation data

To compare methods without touching the mission controller, a separate supervisor teleports the robot to each station, re-assigns the station textures and saves the camera image. Eight cyclic layouts place every target at every station. Two random-pose sets serve as test data: *normal* (up to ±6 cm and $\pm15°$; 512 target and 120 distractor frames) and *hard* (up to ±12 cm and $\pm30°$, where the poster sometimes leaves the frame). Two yaw/offset sweeps (17k target frames, full and lite rendering) measure pose tolerance. This supervisor builds the offline test set only. The submitted model is trained on the eight reference images alone and never sees a simulator frame.

3.3. Encoder comparison

We compared zero-shot text matching, class prototypes from frozen features, and fine-tuning, over 14 encoders. Table 2 shows representative rows, Fig. 3 the per-class errors and Fig. 4 the feature geometry; the full comparison of all methods is in Appendix B (Fig. 11). Two findings drove the design:

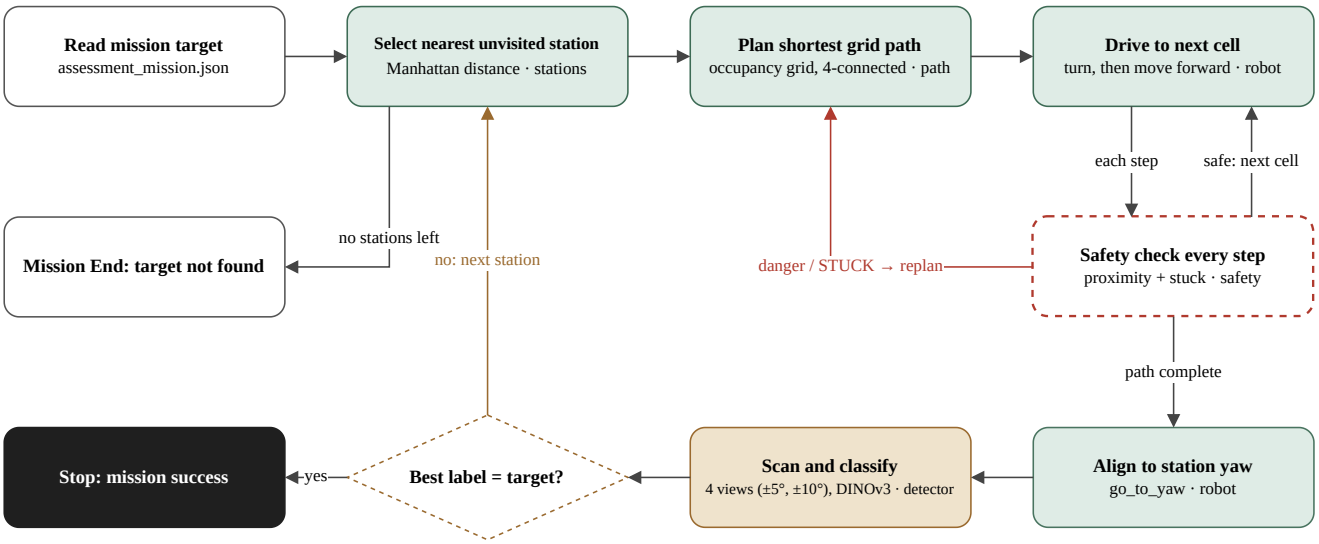


Figure 1. Control flow of the submitted controller, drawn from the main loop: navigation (green), safety checks on every simulation step that stop the move and trigger replanning (red, dashed), and perception at the station (orange). Vision enters only at the final decision; everything before it uses the supplied map, GPS and IMU.

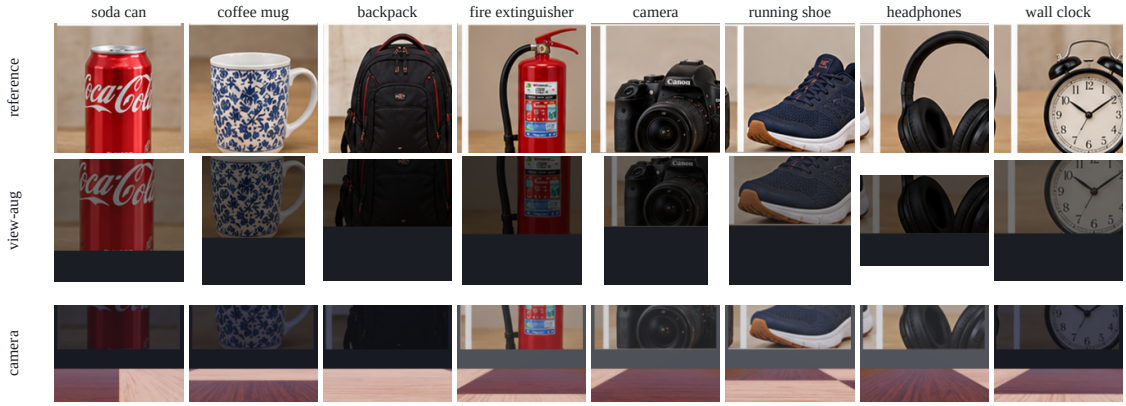


Figure 2. The eight provided reference images (top), view-matched augmentations of them (middle: the lower 40–70% of the reference, brightness ×0.3–0.8, a dark band below), and the e-puck camera frame at each observation pose (bottom). The camera sees only the dark lower half of each poster, so a model trained on the clean references alone faces a large domain gap; the augmentation closes most of it.

- **The first detector's weakness was its data, not its model.** The original ResNet18 [3] was trained on full, bright references with generic augmentation, and it labelled many dark views *running shoe* or *headphones*, the two dark references. The same ResNet18 with view-matched augmentation (Fig. 2, middle) rises from 75.6% to 98.8% on the normal set.
- **CLIP image embeddings do not separate these classes.** With the references as prototypes, CLIP [7] ViT-B/32 reaches 60.0%; the true-vs-nearest-wrong cosine margin is only 0.02–0.03, against 0.43–0.54 for DINOv3 (Fig. 4). CLIP-family models work only in zero-shot *text* mode (MobileCLIP [10]: 95.7%).

3.4. Final model

We fine-tune DINOv3 ViT-S/16 [8] (21.6M parameters; self-supervised on LVD-1689M; timm [11] checkpoint `vit_small_patch16_dinov3.lvd1689m`) on the eight references, with view-matched augmentation and label smoothing [9]. We ship the fine-tuned weights in fp16 (43 MB; predictions identical to fp32). At run time the whole 160×120 frame is resized to 224×224 *without* a centre crop and classified in 32 ms with four CPU threads (91 ms single-threaded). DINOv3 was the most accurate encoder in every setting. Fine-tuning it was also the most stable: across three seeds the s.d. is 0.0 on the normal set, while fine-tuning MobileNetV3 [4] on eight images varied

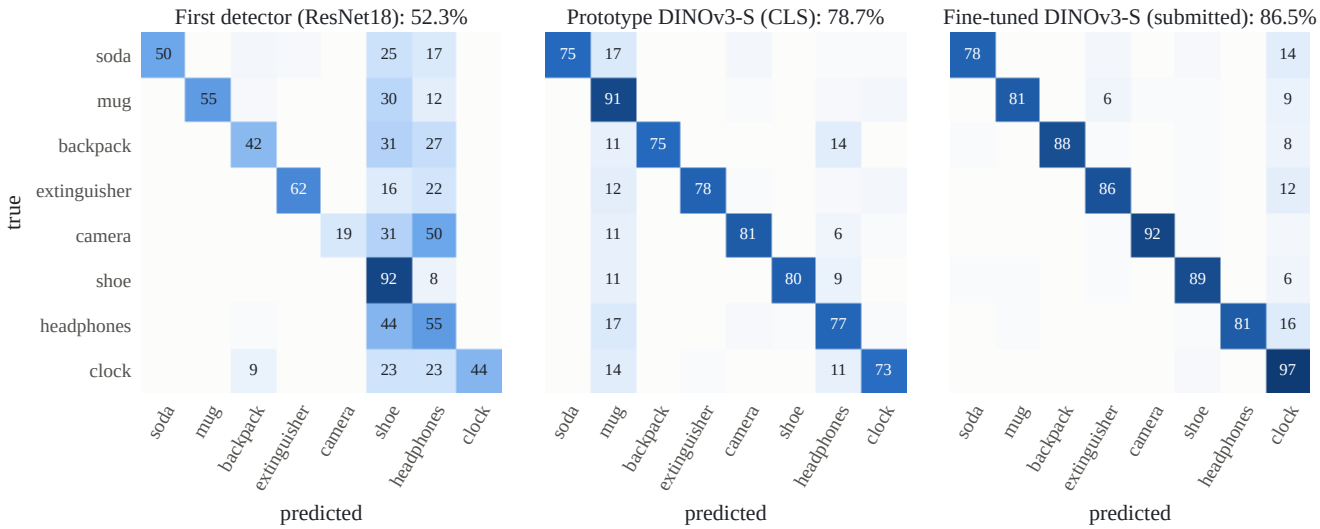


Figure 3. Row-normalised confusion matrices (true vs. predicted, %) on the 512 hard-set target frames for the first detector, the training-free DINOv3-S prototype and the submitted fine-tuned DINOv3-S, with overall accuracy in each title. The first detector sends most dark views to *shoe* and *headphones*, while the fine-tuned model is nearly diagonal; its remaining errors are views where the poster has largely left the frame.

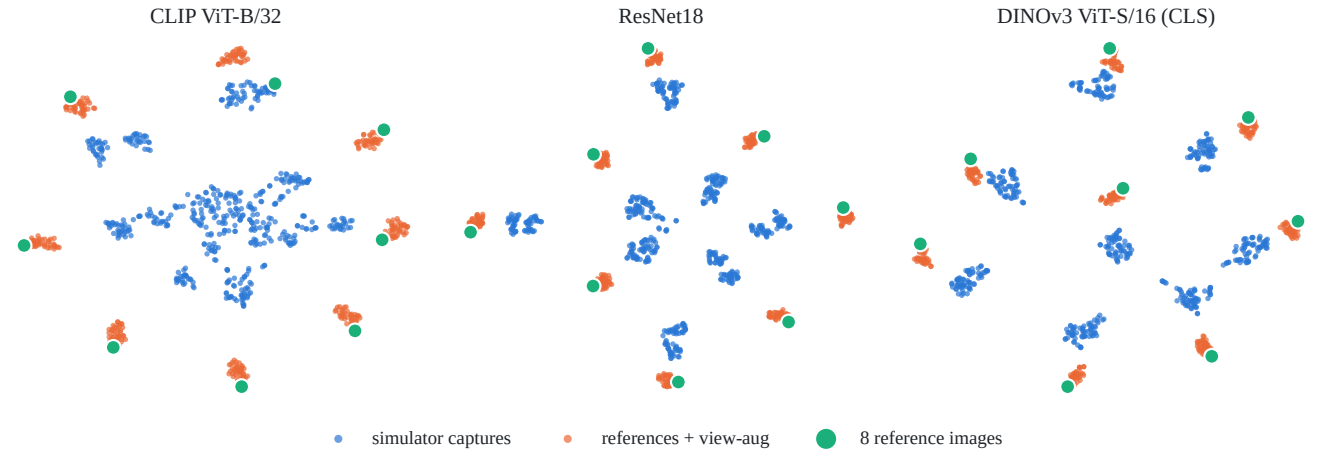


Figure 4. t-SNE of frozen CLIP ViT-B/32, ResNet18 and DINOv3-S (CLS) features for the normal-set captures (blue), the view-augmented references (orange) and the eight original references (green). In DINOv3's feature space the captures and references of each class form one cluster, whereas CLIP's captures collapse into a single mixed blob, which is why CLIP image prototypes fail on this task.

from 55% to 74%.

3.5. Pose robustness and capture policy

The robot rarely arrives perfectly square to the poster, and at a yaw error the poster slides toward the edge of the frame. Two choices address this.

Whole frame instead of centre crop. The standard `CenterCrop` discards the frame edge. Feeding the whole frame raises fine-tuned accuracy at 20–30° yaw error from 71.3% to 92.4% and on the hard set from 81.1% to 86.5%, at no cost. Saliency-based auto-cropping added at most 0.6 points on top for the fine-tuned model, at the cost of a second pass, so we did not use it (Fig. 5).

Multi-view scan. With a single frame, the fine-tuned model keeps $\geq$95% accuracy for yaw errors in $[-30°, +22.5°]$. Rotating through several headings and keeping the most confident frame makes accuracy nearly independent of arrival heading (Fig. 6, Tab. 3). The controller therefore takes four frames at $\pm 5°$ and $\pm 10°$ around the observation yaw. On the benchmark, a single frame proved equally accurate (ablation in Tab. 8: 96/96, 3.5 s faster per mission), because the planner delivers the robot almost square to each station. We keep the scan as insurance against arrival errors beyond

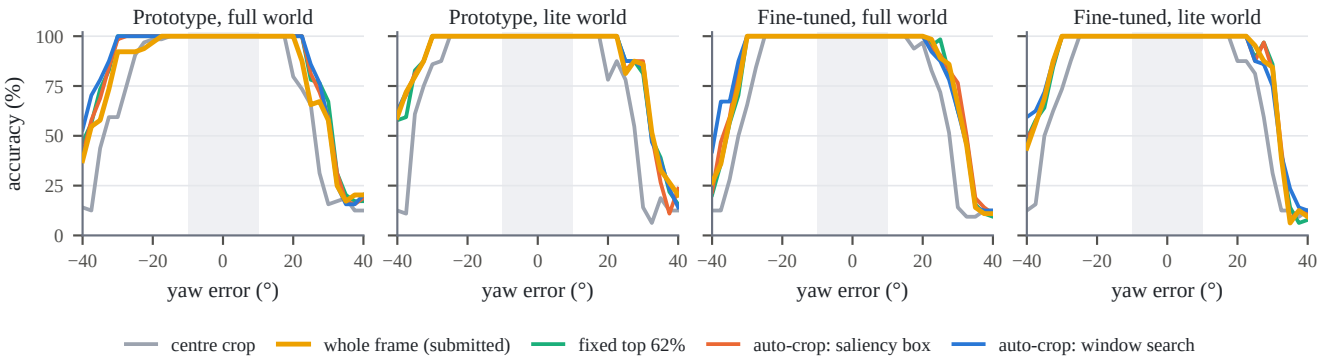


Figure 5. Accuracy against yaw error for five input strategies, for prototype and fine-tuned DINOv3-S in the full and lite worlds, measured on the yaw sweep (8 targets × 8 stations, -40° to $+40^\circ$ in 2.5° steps; grey band: $\pm 10^\circ$). Feeding the whole frame instead of a centre crop recovers most of the accuracy lost at 20–30°, auto-cropping adds little on top, and no strategy recovers views beyond about 30°, where the poster has left the frame.

Method	Encoder	Norm.	Hard	AUROC	ms
First det.	ResNet18	75.6	56.8	0.769	42
Zero-shot	MobileCLIP-S1	95.7	77.7	0.996	143
Proto.+aug	MobileNetV3-L	97.3	78.3	0.989	13
Proto.+aug	DINOv3-S (CLS)	99.0	80.1	0.997	91
FT+aug	ResNet18	98.8	81.6	0.990	42
FT+aug	**DINOv3-S/16**	**100.0**	**88.7**	**1.000**	91

Table 2. Encoder comparison. Accuracy (%) is measured on captured frames from stations the method never saw (normal: up to ± 6 cm/$\pm 15^\circ$; hard: up to ± 12 cm/$\pm 30^\circ$), AUROC on target vs. distractor frames, and latency on one CPU thread; fine-tuned rows are the mean of 3 seeds and every row uses only the eight reference images ("aug": view-matched augmentation; FT: fine-tuned). Fine-tuned DINOv3 is the only method that is perfect on the normal set and clearly best on the hard set, with stable results across seeds (s.d. 0.0 / 0.2).

Encoder	1 frame	3, mean	Scan, best
First (ResNet18)	60.0	60.6	76.2
Prototype DINOv3-S	87.4	96.9	100.0
Fine-tuned DINOv3-S	97.6	100.0	100.0

Table 3. Capture policies, simulated on the yaw sweep: mean accuracy (%) over arrival heading errors in $[-30^\circ, 30^\circ]$ for one frame, three frames at $-10^\circ/0^\circ/+10^\circ$ with averaged scores, and a $\pm 20^\circ$ scan that keeps the most confident frame. Keeping the most confident frame removes almost all heading sensitivity, but only for a calibrated model: the first detector gains far less.

about 20°, for example after a safety recovery.

3.6. Distractors and latency

Barriers B1–B5 carry 20 non-target images. With threshold 0.20 on the softmax confidence, the shipped checkpoint accepts 95% of target frames and none of the distractor frames

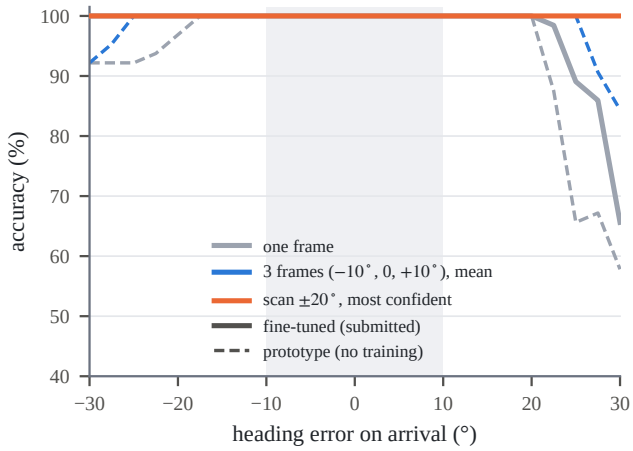


Figure 6. Accuracy against heading error on arrival for three capture policies, simulated by re-scoring yaw-sweep frames around each arrival heading with whole-frame input (solid: the submitted fine-tuned DINOv3-S; dashed: the training-free prototype; the submitted model's 3-frame curve coincides with its scan curve at 100% and is not drawn). Any multi-frame policy removes the submitted model's single-frame drop beyond $+20^\circ$, and keeping the most confident frame of a $\pm 20^\circ$ scan does so even for the weaker prototype.

on the normal set: the 5th percentile of target confidence is 0.215, the 95th percentile of distractor confidence 0.148, AUROC 0.999. The probabilities are deliberately flat because of label smoothing. Confidence also flags the model's own mistakes. On the hard set, all 69 wrong predictions have confidence below 0.157, while 372 of the 443 correct ones exceed 0.20 (Fig. 7b). This is why keeping the most confident frame of a scan works. The mission controller classifies only at observation poses, which always face a station poster, so it makes a closed-set decision and keeps the threshold as a validated safeguard. The four-frame clas-

sification at one station costs about 0.13 s of CPU time. The controller runs in Webots' synchronous mode, so inference does not consume simulation time (Sec. 2).

4. Route Planning

4.1. Problem and design rationale

Robot path planning can be done with many algorithms. Discrete methods such as Dijkstra [1], A* [2] and Bellman-Ford search a graph; continuous sampling methods such as Rapidly-exploring Random Trees (RRT) and RRT* sample the free space. They trade off time complexity, space complexity, support for negative weights and optimality of the path.

In this project the arena is a known 2D map that fits a 40×40 grid, and the obstacles and observation stations are available to the program at run time. The station that shows the target, however, is unknown to the robot, so stations must be visited until the target is found. Navigation therefore reduces to repeated shortest-path queries on a known graph, followed by a visual check at each station.

4.2. Map representation

As mentioned, the obstacles and stations lie within a 40×40 grid at 0.1 m per cell (385 occupied cells). The provided program includes a coordinate system and helpers that convert GPS positions to grid cells and back. The IMU yaw, rounded to the nearest multiple of $90°$, gives the robot's grid heading. The planner rejects the two outermost rows and columns as a safety margin along the arena wall.

4.3. Shortest path

Given the problem and environment, we narrowed the choice to discrete methods, in particular Dijkstra and A*.

Why not sampling-based planners. Sampling methods do not match the cell-based coordinate API we were given. They could arguably produce shorter paths, because they allow motion in any direction rather than the four directions and discrete cells we chose, but they are non-deterministic and need path smoothing. With the coordinate API and a generous 4-minute limit, we chose the simpler discrete approach. Incremental replanners such as D* Lite were also unnecessary: replanning happens only after a safety event, and a full replan takes well under a millisecond.

Why A*. Among discrete methods we required a guaranteed shortest path. Depth-first search (DFS) gives no such guarantee and was excluded. Breadth-first search (BFS) does return shortest paths on our grid, where every move costs the same; in fact our Dijkstra on unit costs behaves like BFS with a priority queue. However, BFS stops being optimal as soon as moves have different costs, for example turn penalties or clearance costs, so we preferred a cost-based planner that extends to them. Bellman-Ford was ruled out because our costs are never negative, so its extra complexity buys nothing. Dijkstra and A* both return optimal paths when the A* heuristic never overestimates the true distance, which holds for the Manhattan distance on a 4-connected grid. We therefore chose between them by execution time (Tab. 4).

Time (ms)	Mean	Median	Min	Max
Dijkstra	0.757	0.764	0.144	1.410
A*	0.208	0.153	0.026	0.767
Speed-up	$3.65\times$ (by mean)			

Table 4. Planner execution time over all 110 ordered start/station pairs on the 40×40 occupancy grid (25 repeats per pair, one CPU core). Both planners returned paths of identical, optimal length on every pair, and A* is $3.65\times$ faster by mean time (median per-pair speed-up $4.2\times$, range 1.7–$10.3\times$; Appendix A).

As Tab. 4 shows, A* runs $3.65\times$ faster than Dijkstra by mean time, and the per-pair median speed-up is $4.2\times$. We therefore chose A*. Both planners finish in under a millisecond, while a mission takes 70–160 s, so the speed-up has little effect on mission time. Appendix A adds greedy best-first search, which is faster still but returns longer paths on 26 of 110 pairs, and a turn-penalised A* that keeps paths optimal while needing 47% fewer turns.

4.4. Station visiting order

For the visiting order we chose a greedy nearest-neighbour rule. The planner keeps a list of all station coordinates and pops the nearest one (Manhattan distance from the robot's cell) as the next destination. TSP-style tours also exist, but the mission can end at any station before all have been visited, which makes nearest-neighbour search appealing. The resulting tours are fixed by the start position (Fig. 8). Their cost shows in the student missions: from every start the target station S1 is reached only 7th or 8th, which sets their 130–162 s mission times (Fig. 9).

We also considered an information-driven approach. An object detector would find station images anywhere in the robot's view, and stereo mapping would project each detection to world coordinates using the camera intrinsics and extrinsics, locating the station it belongs to. That station would then be removed from the list if it did not show the target, or visited directly if it did. However, the robot has no stereo or depth camera, so world coordinates cannot be recovered from a single detection.

4.5. Motion execution

Once a path is generated, the robot receives a list of discrete moves in the world frame. The problem then becomes turning a world direction such as "UP" into in-place rotations and forward moves. We solve it with rotation matrices: one

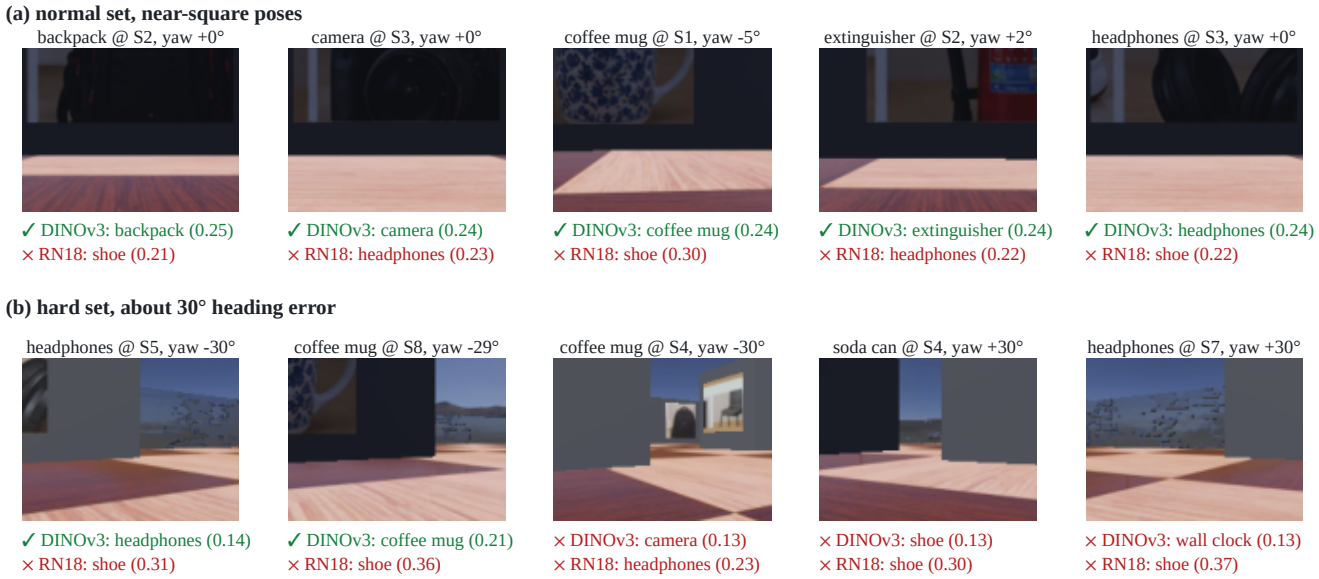


Figure 7. Predictions of the submitted DINOv3 detector and the first ResNet18 detector (RN18) on captured camera frames (true label, station and heading error above each frame; predicted label and softmax confidence below; "shoe" is *running shoe*). (a) At near-square poses the first detector maps dark posters to *running shoe* or *headphones*, while DINOv3 is correct. (b) At about 30° heading error DINOv3 can still succeed, and it fails only when the poster has largely left the frame, always with low confidence ($\leq$0.16), which the scan and the 0.20 threshold exploit.

for the desired global direction and one for the robot's current global direction. The goal is the rotation the robot must perform to face the desired direction.

This is similar to the extrinsic matrix in computer vision: we need the world direction as seen from the robot. The inverse of the robot's global direction serves as its extrinsic matrix. Multiplying it by the desired global direction gives that direction in the robot's frame, which is exactly the rotation the robot needs (straight, left, right or back).

Each forward move drives at full wheel speed until the GPS cell changes, while the safety layers (Sec. 5) monitor every simulation step. Each turn rotates in place until the IMU yaw is within 0.1 rad of the next multiple of 90°. At the station, `go_to_yaw` turns the shorter way to the observation yaw, and the robot takes four frames at $\pm 5°$ and $\pm 10°$ for the classifier (Sec. 3). Raising the scan's wheel speed from 0.1 to 0.5 rad/s, released together with the switch to A*, cut the worst-case mission time from 229 s to 162 s (Tab. 7).

5. Safety

5.1. Problem and design rationale

Even a single collision causes a mission to fail. Our path planner uses an occupancy grid to generate a path that avoids obstacle cells, but it cannot account for heading errors, deviations in travel distance, and contact with or slippage along walls that occur during actual driving.

Consequently, instead of making changes to the path planner itself, we created an additional monitoring layer—a safety function that observes the robot's actual state while the planned movement takes place. When danger is detected, the current movement is stopped immediately, and a safety event is returned to prompt the controller to replan the path from the current position.

There are two reasons why a monitoring-based approach is preferable to a reactive obstacle avoidance method. First, in a static assessment environment, since the global planner already provides a valid grid path, continuous reactive steering could disrupt the planned path and the discrete one-cell movement structure. Second, the monitoring layer does not change the navigation behaviour unless there is an anomaly and, since the checks are solely computational and Webots waits for the controller at every step, it does not add to the mission time.

The safety system consists of two complementary layers. The first layer uses proximity sensors to detect a dangerous approach to obstacles before a collision occurs. The second layer is a stuck detector that monitors actual movement and serves as a fallback mechanism to detect contact that the proximity sensors may have missed.

5.2. Layer 1: directional proximity monitoring

Proximity monitoring of four sensor groups (FRONT, RIGHT, BACK, LEFT; Tab. 5) is carried out at each simulation step (32 ms) for each forward movement. If one

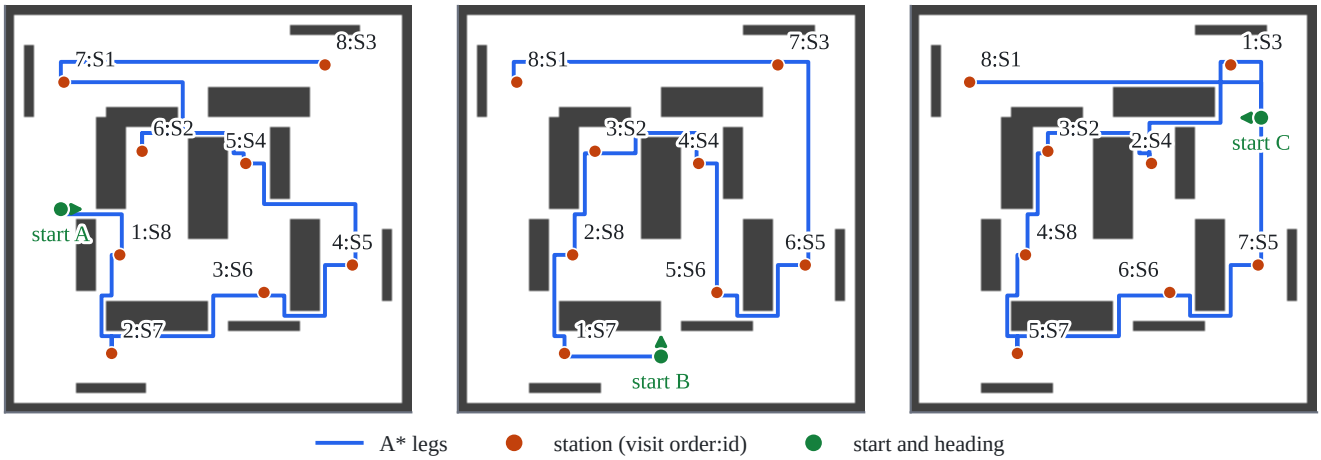


Figure 8. Greedy nearest-neighbour tours from (a) Start A, (b) Start B and (c) Start C, produced by replaying the controller's station order and A* planner on the 40×40 occupancy grid (blue: A* legs between consecutive observation cells; labels: visit order and station; green arrow: start pose). The order is fixed by the start alone, and the mission stops at the first station showing the target, so mission time depends mainly on that station's position in the tour: S1, the student-mission target, comes 7th or 8th from every start.

or more sensor groups exceed their threshold values, the movement stops immediately, resulting in a safety event being activated. To handle situations of simultaneous danger detection from different directions, a set is returned, representing the dangers detected. Proximity monitoring is not carried out when the robot performs a rotation on the spot because the robot's circular footprint means there is no difference in the distance of its center from any obstacles as it rotates in place.

The front threshold was determined through a calibration test. Using a test-only controller (`safety_diagnostic`), the robot was driven toward a wall at the same speed as in an actual mission; the sensor readings and distance traveled for each step were recorded until contact was made, and the remaining distance (clearance) for each step was calculated in reverse from the point of contact. The peak values of all proximity sensors observed during normal full-mission operation (Start A/B/C) were 80.4 or lower. The initial front threshold of 80 lies within this noise range, which confirms that the current threshold of 120 is needed; it provides a sufficient margin between normal movement and dangerous proximity. The side threshold (150) and back threshold (120) remain at their initial values; while it was confirmed that there were no false positives during normal operation, unlike the front threshold, no separate calibration for clearance was performed.

In the initial implementation, only ps0 and ps7 were used for detection of front dangers. However, during a calibration test involving an approach to a wall at a 45° angle, it was observed that the front-diagonal sensor ps6 exceeded the threshold before ps7 did. It was concluded that detection was delayed when using only ps0 and ps7 in approaching the wall at an angle; thus ps1 and ps6 were added as front-diagonal sensors. After this addition, the clearance at a stop during a 45° approach increased from 3.6 cm to 5.2 cm, while the clearance during a head-on approach remained at the original 2.5 cm.

If only a FRONT danger is detected, the robot verifies whether the back sensors indicate a safe condition and then performs a brief reverse recovery, and continues to monitor for BACK proximity even while reversing. In the case of LEFT, RIGHT, or BACK danger, the robot does not perform a separate reactive avoidance manoeuvre.

5.3. Layer 2: progress-based stuck detection

Since the existing movement code is designed to keep the robot moving forward until it enters a new grid cell, if the robot becomes physically blocked without the proximity layer detecting the danger, the same forward command may continue for an extended period. To prevent this issue, we implemented a second safety layer that monitors the actual distance traveled using GPS location data.

The theoretical distance traveled per step when moving forward normally is

$$d = \omega r \Delta t = 6.27 \times 0.02 \times 0.032 \approx 0.0040\ \text{m} = 4.0\ \text{mm}, \tag{1}$$

so it takes approximately 25 simulation steps to move one 0.1 m grid cell. Based on this, we use two STUCK detection rules.

- **Rule A:** If none of the last 8 steps involved a movement exceeding 3.0 mm, the robot is deemed STUCK.
- **Rule B:** If a single forward move exceeds 75 steps but the robot fails to enter a new grid cell, it is deemed STUCK.

Parameter	Value	Rationale
Front (ps0/ps7)	120	2.5 cm clearance in head-on approach; initial value of 80 lies within normal noise (max 80.4)
Front-diagonal (ps1/ps6)	120	5.2 cm clearance in a 45° approach (3.6 cm before adding ps1/ps6)
Left/right (ps5/ps2)	150	Initial value; no false positives; clearance not calibrated
Back (ps3/ps4)	120	Initial value; used only during reverse recovery
Max. normal-driving reading	≤ 80.4	All eight sensors, complete missions from Start A/B/C
Expected displacement per step	4.0 mm	$6.27\,\mathrm{rad/s} \times 0.02\,\mathrm{m} \times 32\,\mathrm{ms}$
Slow-step threshold	3.0 mm (75%)	Separates normal motion from wall sliding (1.8–2.3 mm/step)
Stuck-detection window	8 steps	Twice the maximum of four consecutive slow steps in normal driving
Max. steps per cell	75	Three times the typical ≈25 steps
Reverse recovery	−3 rad/s, 4 steps (≈8 mm)	Only after confirming that the back is safe

Table 5. Safety parameters and their calibration rationale.

If a STUCK event is detected, the system checks whether the back is clear—just as it does during FRONT recovery in the proximity layer—and then initiates a short reverse recovery.

The detection thresholds were established based on actual driving logs. While normal free motion is approximately 4.0 mm/step, irregular displacements ranging from approximately 1.9 to 4.0 mm could occur immediately after rotation. However, during normal operation, such slow steps were observed for a maximum of four consecutive steps. In contrast, in the wall-contact calibration runs, step displacements remained at 1.8–2.3 mm when sliding along a wall and ≈0 mm when blocked head-on. Therefore, the 8-step window was set to provide a margin long enough to account for normal temporary speed fluctuations while detecting actual movement failure within approximately 8 steps—that is, within about 0.26 s.

5.4. Safety results

- **Normal missions:** In all runs (Start A/B/C), the maximum proximity reading across all eight sensors was 80.4 or lower. The Layer 1 safety mechanism was never triggered, and there were no false positives at Layer 2. Therefore, the safety layer did not add any time overhead to the normal mission.
- **Fault injection (`stuck_test`):** The robot was driven into a wall while the proximity sensor values input to Layer 1 were intentionally blocked. Layer 2 detected a STUCK condition in both head-on and 45° approaches. Since the actual sensor values at the time of stoppage (373, 993) significantly exceeded the threshold of 120, it was confirmed that the detection occurred in an actual contact situation.
- **Benchmark:** The final version had zero collisions in 96 Monte Carlo runs. However, the previous version (e97b0e5), which lacked safety monitoring during movement, also had zero collisions in the same benchmark. Therefore, it is reasonable to evaluate the safety layer not as the cause of passing the benchmark, but as a defence-in-depth mechanism designed to prepare for situations that cannot be predicted during the planning phase. Logged event counts confirm this: 0 proximity events and 0 stuck detections in 192 nominal missions, while with an unmapped box on the route the layer stops the robot in every run and raises success from 0% to 59% (Sec. 6.5).

World	Result	Time (s)	Final dist. (m)
Start A	success	130.2	0.033
Start B	success	140.6	0.039
Start C	success	161.5	0.038

Table 6. The three provided student missions run unchanged with the final controller (target `soda_can` at S1; time in Webots simulation seconds; distance from the final pose to the observation position, scored by the referee). All three succeed with at least 78 s of margin and stop within 4 cm of the goal.

6. System Evaluation

6.1. Protocol

We evaluate the full system in two ways. *(i) Student missions:* the three provided worlds `training_start_{A,B,C}` as supplied, with the supplied mission target (`soda_can`, displayed at station S1). *(ii) Monte Carlo benchmark:* 3 starts × 8 targets × 4 station layouts = 96 missions per controller version. A layout is a cyclic re-assignment of the eight target images to the eight stations, applied by a supervisor before the robot moves, so each target is searched for at several stations and from every start. All runs use headless Webots R2025a on a CPU cluster. An independent referee supervisor, which never communicates with the controller, scores each run against the assignment criteria: success requires stopping within 0.20 m of the correct observation position, with no contact other than the wheels on the floor, within 240 s of simulation time. The referee also records completion time, final distance and the station the robot stopped at. The controller uses no ground truth.

6.2. Results

The final controller completes all three student missions (Tab. 6) and all 96 benchmark missions (Tab. 7) with no

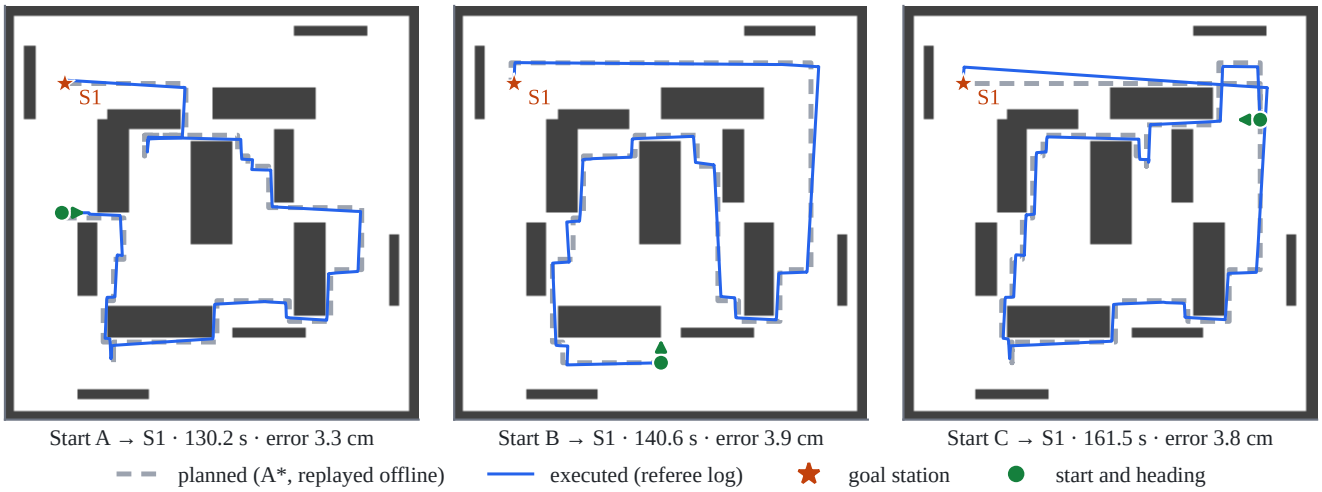


Figure 9. Planned versus executed trajectories for the three student missions: the referee-logged trajectory (blue) against the A* route obtained by replaying the controller's station order and planner offline (grey, dashed), with each mission's completion time and final distance to the S1 observation position below the panel. The executed path follows the plan from all three starts (median deviation 2–3 cm, at most 16 cm on Start C's final straight), and all missions end at S1 within 130–162 s with final errors below 4 cm.

Version	Success	Coll./TO	Time (s) med/p90/max	Dist. (m) med/max
V0	0/96	92/4	102/179/240	1.881/4.424
V1	96/96	0/0	73/143/167	0.110/0.157
V2	96/96	0/0	108/197/229	0.038/0.077
V3 (final)	**96/96**	**0/0**	**73/141/162**	**0.037/0.076**

Table 7. Monte Carlo benchmark of four controller versions on the same 96 missions (3 starts × 8 targets × 4 station layouts, scored by the referee). V0: first integration; V1: motors stopped after the mission; V2: no step-back, 4-frame scan, safety layers; V3: A* and a 5× faster scan (submitted; also 192/192 over all 8 layouts, Tab. 9). Coll./TO: collisions/timeouts. Every version found the correct station; the gains came from stopping correctly (V1), stopping precisely (V2) and restoring the time margin (V3).

collision. The slowest mission takes 162 s, leaving a 78 s margin to the time limit; per start, the median and maximum times are 80/157 s (A), 70/141 s (B) and 73/162 s (C). The student missions are slower than the benchmark median because S1 comes late in every start's tour (Fig. 8). Figure 9 checks that the robot executes what the planner intends: the logged trajectory stays within a median 2–3 cm of the replayed A* route, and the largest deviation, 16 cm, builds up on Start C's 1.6 m final straight, where heading drift is not corrected within a move.

Table 7 also shows how the system developed. Every version, including V0, *found* the correct station in all 96 runs (median closest approach 0.036 m). The differences come entirely from what happens after recognition. V1 fixed a stop bug that turned V0's 96 correct identifications into 0 successes (see below). Removing the step-back in V2 brought the median final distance from 0.110 m to 0.038 m, but the slow scan raised the worst case to 229 s, only 11 s inside the limit. V3 restored the time margin and kept V2's precision (Fig. 10).

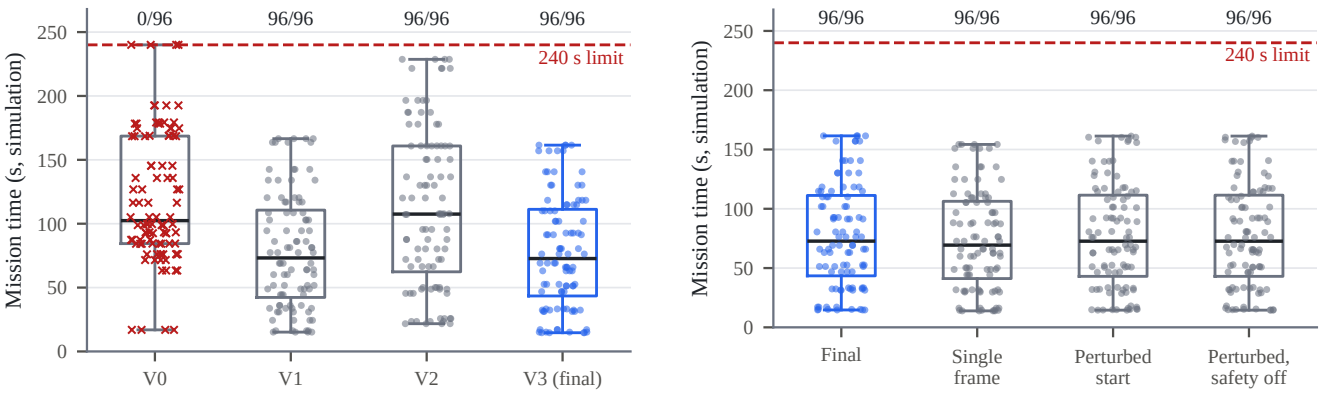
6.3. Failure cases

Collisions after a correct identification (V0). V0 identified the target correctly in every run but failed all 96: 92 by collision and 4 by timeout. After recognition it called a step-back routine that set both wheels to reverse and returned *without* stopping them. Because `main()` then returned, the motors kept their last command, and the robot reversed across the arena until it hit an obstacle (median final distance 1.88 m). Stopping the motors and idling the controller after the mission (V1) fixed every run.

Detector confusion. The first detector, a ResNet18 classifier, confused `backpack` with `running_shoe` in early integration tests. Re-scoring it on the captured frames shows a systematic bias rather than a single mistake. Its most frequent errors send dark posters to *headphones* or *running shoe* (camera→headphones 79 times, headphones→shoe 49, backpack→shoe 40, over 1,024 frames), even at square poses (Fig. 7a). This motivated the encoder study and the view-matched augmentation of Sec. 3.

Near-timeout runs. V2's slow scan pushed 8 of 96 missions beyond 200 s. No run failed, but the margin was too small for a live demonstration, which led to the faster scan in V3.

Stuck-detector false positives. The initial stuck detector



(a) Controller versions (Tab. 7)

(b) Ablations of the final controller (Tab. 8)

Figure 10. Mission time of every benchmark run (96 per box; box: quartiles; dots: successes; red crosses: failures; success count above each box; referee-scored Webots simulation time). (a) V0 fails every run despite correct identification, and V3 combines V1's speed with V2's precision. (b) None of the ablations changes the outcome: the time distributions are almost identical and every run stays far below the 240 s limit.

classified the robot as STUCK if the cumulative displacement over the last 10 steps was less than 70% of the expected displacement. However, this approach generated 10 false positives during the Start A/B/C normal missions. Analysis of debug logs showed that immediately after turning to a heading of $\pm\pi$, even during normal operation, the step displacement alternated between approximately 2 mm and 4 mm, and the cumulative criterion misinterpreted this temporary fluctuation as a stop. Nevertheless, during normal operation, such slow steps occurred at most four times in a row. Accordingly, the decision rule was redesigned to check whether any single step among the most recent 8 steps exceeded 3.0 mm (Rule A in Sec. 5), rather than using the cumulative criterion. After the modification, the number of false-positive STUCK events during normal missions decreased to 0, and STUCK detection was maintained for both head-on and 45° approaches during fault-injection tests with the proximity sensor input disabled.

6.4. Ablations

Table 8 isolates three design choices on the same 96 missions. *Single frame:* the 4-frame scan is replaced by a single frame at the observation yaw. *Perturbed start:* the supervisor shifts the start pose by up to ± 5 cm in x and y and $\pm 15°$ in heading (uniform, fixed seed per run). *Safety off:* the same perturbed missions with proximity monitoring and stuck detection disabled.

Scan. A single frame is as accurate as the 4-frame scan on all 96 missions and saves 3.5 s per mission (paired median). The planner delivers the robot almost square to each station, within the single-frame tolerance of Sec. 3. We keep the scan because it costs little and covers larger arrival errors, where single-frame accuracy drops (Fig. 6).

Variant	Success	Failures	Time (s) med/max	Dist. (m) med
Final (V3)	96/96	0	73/162	0.037
Single frame	96/96	0	69/154	0.038
Perturbed start	96/96	0	73/161	0.038
Pert., safety off	96/96	0	73/161	0.038

Table 8. Ablations on the same 96 missions, each changing one factor of the final controller (perturbed starts: up to ± 5 cm and $\pm 15°$, same random seed per mission in both perturbed rows; referee-scored; failures: collisions, timeouts and wrong stops). The system is insensitive to all three changes on this benchmark, so the scan and the safety layers act as insurance for conditions the benchmark does not produce rather than as sources of the measured success.

Start perturbation. Shifting the start pose changes the mission time by a paired median of -0.4 s (maximum $+1.0$ s). The first 90° turn snaps the heading back to the grid, and GPS cells absorb the position offset.

Safety off. With proximity monitoring and stuck detection disabled, all 96 perturbed missions are identical to the safety-on runs, to 0.01 s of simulation time. The event logs confirm this directly: across 192 nominal missions over all eight layouts, the controller logged 0 proximity events and 0 stuck detections (Tab. 9). The planned paths alone kept every nominal run collision-free. The safety layers are therefore headroom for disturbances outside this benchmark (unmapped obstacles, wheel slip, a mis-set start). Section 5 evaluates them directly with fault injection.

Condition	Success	Coll.	Wrong	TO	Events	Stuck
Nominal	192/192	0	0	0	0 (0)	0
Box, safety off	0/95	95	0	0	–	–
Box, safety on	55/93	18	20	0	11,504 (93)	0

Table 9. Nominal benchmark over all eight layouts (3 starts × 8 targets × 8 layouts) and the unmapped-obstacle test (one box on the first planned route, same box per seed with safety ON and OFF), scored by the referee; Events: logged proximity stops (runs with at least one in brackets); Stuck: logged stuck detections; TO: timeouts. Three safety-on runs and one safety-off run, all from Start A, where the robot kept pressing against the box, exceeded the cluster's wall-clock limit and are not included. Without safety every obstacle run collides, with safety most succeed, and in nominal runs the safety layers never fire.

6.5. Unmapped obstacle and safety events

To test the safety layers where they matter, the referee places a $6\times6\times8$ cm box that is *not* in the occupancy grid on a cell of the robot's first planned route, before the robot moves. It replays the controller's planner, draws the cell with the run's seed, and uses the same box in the ON and OFF runs. Safety events are counted from the controller's own log lines. We also re-ran the nominal benchmark over all eight layouts, so every target appears at every station.

Nominal, all layouts. All 192 missions succeed (median 73 s, worst 162 s), with no safety event. With deterministic physics, mission time depends only on the start and the goal station, so layouts 4–7 reproduce the time distribution of layouts 0–3 exactly; vision never changes the path.

Safety off. The planner drives straight into the box: every run collides, after a median of 9.7 s.

Safety on. The proximity layer stops the robot in front of the box in every run, and 55 of 93 completed missions (59%) still succeed. The logs show how. After each FRONT event the robot reverses and replans, but the static map yields the same route, so it stops again. This repeats until the 100-retry limit (median 100 events per run), after which the robot abandons that station and continues the tour. The two remaining failure modes match the limitations in Sec. 7. In 20 runs the robot, having given up, classifies the view from wherever it stands; because the closed-set decision always returns a label and the 0.20 threshold is not applied in the loop, it accepts a wrong station. The other 18 runs, all from Start A, end in contact after about 200 repeated reverse-and-replan cycles. Stuck detection never fired: every event was caught by the proximity layer first.

7. Limitations and Future Work

7.1. Limitations

Evaluation scope. All runs use the supplied training worlds, software rendering without shadows, and a deterministic simulator. The benchmark varies start, target and station assignment, plus a ±5 cm/$\pm15^\circ$ start perturbation, but not lighting, rendering quality or wheel slip. Every nominal and perturbed run succeeds, so only the unmapped-obstacle test (Sec. 6.5) shows where the system breaks.

Unmapped obstacles. The obstacle test quantifies the first safety limitation below: replanning on the static map sends the robot back to the same blocked cell until the retry limit, and accepting a closed-set label at a wrong pose then causes wrong stops (Tab. 9).

Perception. The detector is trained on eight images and tested on frames from the same world. Accuracy falls once the poster leaves the frame (single frame: 65.6% at $+30^\circ$; Fig. 6), and a darker GPU renderer on the assessment machine could shift confidences, because darkness was the main nuisance factor in our tests. The 0.20 confidence threshold is validated offline but not used in the loop.

Planning and time. Paths are 4-connected, so a route turns 2.9 times on average, and greedy ordering reaches some stations last regardless of where the target is (S1 is 7th or 8th from every start). The worst benchmark mission uses 162 of 240 s, so the remaining margin depends on the target's position in the fixed visiting order.

Safety. First, replanning after a safety event uses the same occupancy map. Therefore, if an event is caused by an obstacle that is not present in the map, the planner may repeatedly generate the same route until the retry limit is reached. Temporary blocking of repeatedly problematic cells was not implemented because no safety events occurred during normal missions, the assessment environments match the occupancy grid, and additional detours could reduce the available time margin. Second, LEFT, RIGHT, and BACK dangers prompt the robot to stop and replan, but no direction-specific recovery manoeuvre is performed. Third, the side and back proximity thresholds have been checked for false-positive occurrences but were not calibrated against stopping clearance. The front threshold has been calibrated in a more direct way; however, it was calibrated only against the tested wall-approach geometries in simulation, limiting its applicability to other shapes or approach angles.

7.2. Future work

Fewer turns. A turn-penalised A* finds equally short paths with 47% fewer turns (Appendix A). Since a 90° turn costs about as much time as driving one cell, this is the cheapest way to shorten missions. A pure-pursuit or PID follower on a smoothed path would go further.

Confidence-gated scan. A single frame is as accurate as the four-frame scan on the benchmark and 3.5 s faster (Tab. 8). Scanning only when the first frame's confidence is below the 0.20 threshold would keep the insurance against large arrival errors at almost no cost.

Handling unmapped obstacles. Two small changes target both obstacle failure modes. Marking the cell in front of the robot as occupied after a FRONT event would make the planner route around the box instead of retrying the same path. Applying the validated 0.20 confidence threshold at the station would turn a low-confidence view from a wrong pose into "not the target". All 69 wrong predictions on the hard set fall below it (Sec. 3).

Learned local control. The global problem is solved optimally by planning on a known map, so reinforcement learning would not improve route choice. Where it could help is local motion: a continuous-action policy (SAC or TD3) that outputs wheel speeds, or a residual correction on top of the current move/turn primitives, trained in simulation and compared against the hand-written controller on the same benchmark. We did not adopt it because the current controller already succeeds on every benchmark mission, and a learned policy would be harder to verify against the time and collision constraints.

Safety. Future improvements could include temporarily marking cells associated with repeated safety events as occupied before replanning, adding direction-specific recovery behaviours for side and back dangers, and calibrating the side and back thresholds using controlled shallow-angle and reverse-approach tests.

8. Conclusion

Separating navigation on the known map from visual verification at each station gives a controller that is simple to reason about and reliable in our tests: all three student missions and all 96 benchmark missions succeed with no collision, a median stopping error of 3.7 cm and a worst case of 162 s. The largest gains came from understanding the data, namely the low, dark camera view, and from testing the integrated system end to end, which exposed the motor bug that turned 96 correct identifications into 0 successes.

Contribution Statement

Member	Responsibilities
Syuan-Hao Li	Perception (encoder study, DINOv3 fine-tuning, detector integration); Monte Carlo benchmark and referee; **[TODO: report sections]**
Isaac Mangano	Route planning, robot motion, main controller loop and integration; station scan; **[TODO:]**
Hyunkyung Lee	Safety monitoring and recovery; proximity-threshold calibration; progress-based stuck detection; diagnostic and fault-injection testing; evaluation logging and integration validation.

[TODO: How the work was integrated: shared repository, pull requests and review, Monte Carlo runs on each merged version.]

References

[1] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. 6

[2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. 6

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 3

[4] Andrew Howard, Mark Sandler, Grace Chu, et al. Searching for MobileNetV3. In *ICCV*, 2019. 3

[5] Olivier Michel. Webots: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1 (1):39–42, 2004. 1

[6] Francesco Mondada, Michael Bonani, Xavier Raemy, et al. The e-puck, a robot designed for education in engineering. In *Proceedings of the 9th Conference on Autonomous Robot Systems and Competitions*, 2009. 1

[7] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 3

[8] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 3

[9] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016. 3

[10] Pavan Kumar Anasosalu Vasu, Hadi Pouransari, Fartash Faghri, Raviteja Vemulapalli, and Oncel Tuzel. MobileCLIP: Fast image-text models through multi-modal reinforced training. In *CVPR*, 2024. 3

[11] Ross Wightman. Pytorch image models. `https://github.com/huggingface/pytorch-image-models`, 2019. 3

A. Planner comparison

We compare four planners offline on the 110 ordered start/station pairs from the route-planning benchmark (11 points: three starts and eight observation cells). Dijkstra and A* are the controller's own implementations; greedy best-first search (GBFS) orders the frontier by the Manhattan heuristic alone; turn-penalised A* searches over (cell, heading) with cost = cells + w·turns, $w = 1$, since a $90°$ turn takes about as long as driving one cell. Timing is the median of 25 repeats on one CPU core. We checked all 440 paths for validity (adjacent free cells, consistent move labels).

Table 10 shows three things. First, optimality matters: GBFS is the fastest planner but returns a longer path on 26 of 110 pairs, up to $2.29\times$ the optimum, and needs 18% more turns than A*. Second, A* and Dijkstra give paths of identical length; A*'s $4.2\times$ speed-up is real but irrelevant to mission time, because every query finishes in under a millisecond. Third, the metric that does affect mission time is the number of turns. Among equally short paths, the turn-penalised variant needs 47% fewer turns than A* (170 vs. 324). It is not used in the submitted controller and is left as future work (Sec. 7).

B. Full encoder comparison

Figure 11 extends Tab. 2 to every method of the encoder study, with simulated mission success as a third metric.

Planner	ms (median)	vs. Dijkstra (speed-up)	Optimal pairs	Length ratio mean / max	Turns total (mean)
Dijkstra	0.637	1.0×	110/110	1.000 / 1.00	308 (2.8)
A* (final)	0.127	4.2×	110/110	1.000 / 1.00	324 (2.9)
GBFS	0.060	10.3×	84/110	1.059 / 2.29	383 (3.5)
A* + turn penalty	0.450	1.2×	110/110	1.000 / 1.00	170 (1.5)

Table 10. Offline planner comparison on all 110 ordered start/station pairs (median time of 25 repeats on one CPU core; path length relative to the Dijkstra optimum; turns are heading changes along the path). Optimality matters (GBFS is longer on 26 pairs), planning speed does not (all under 1 ms), and turn count is the cost that remains to optimise.

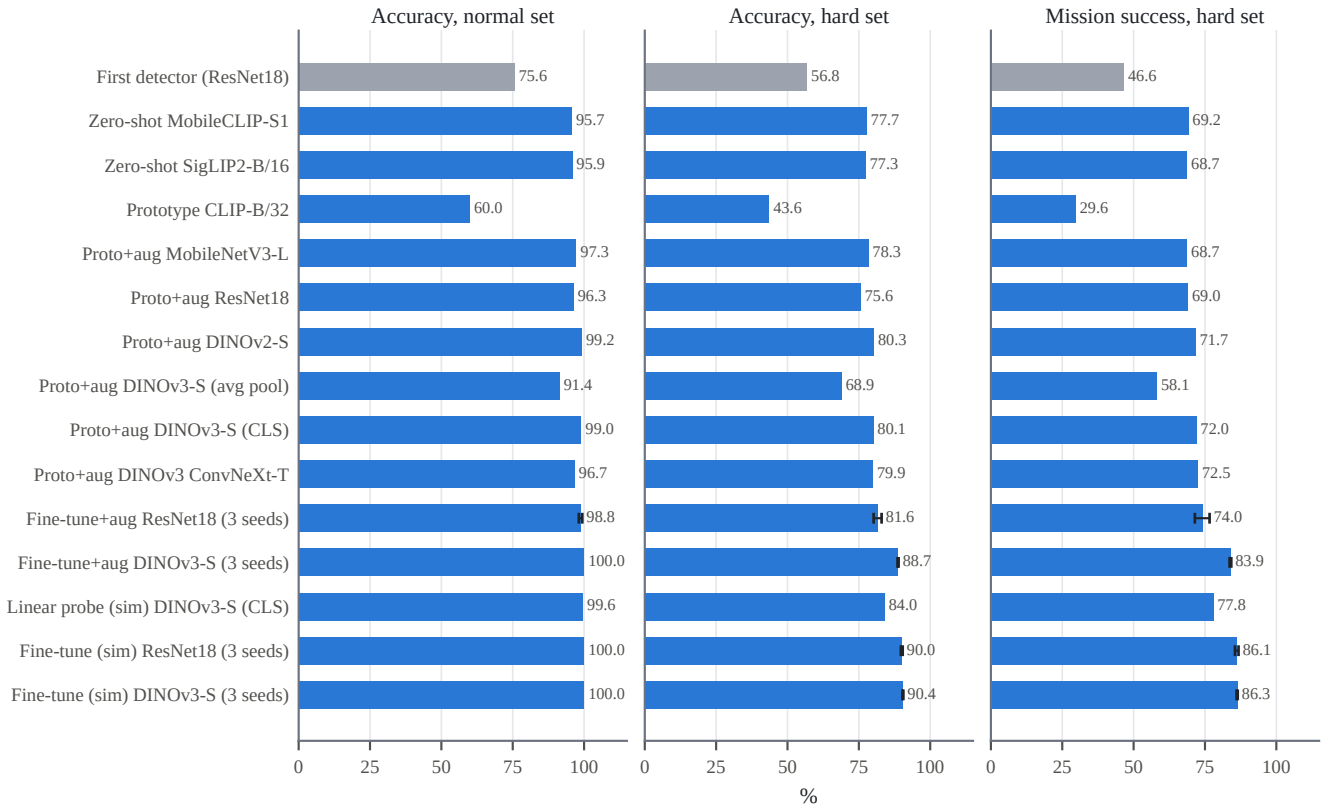


Figure 11. Accuracy on the normal and hard capture sets and simulated mission success (random station order, stop at the first positive, success if that station is correct) for every method, measured on frames from stations the method never saw; fine-tune bars show the mean and s.d. of 3 seeds. Training-free DINOv3 prototypes already beat the first detector by 23 points on the hard set, and fine-tuning DINOv3 on the eight references gives the best accuracy (88.7%) and mission success (83.9%) without any simulator data.